Next.js & TypeScript & API

1. Typescript cho React

a. Cơ bản về Type & Interface:

i. Cách định nghĩa interface cho Object.

- Interface đóng vai trò như là blueprint để định nghĩa hình dáng hoặc cấu trúc của 1 object, ép buộc object phải tuân thủ đúng cấu trúc dữ liệu được dề ra
  → Tránh lỗi gõ nhầm / lỗi thiếu trường dữ liệu
- Cách định nghĩa cơ bản:

```
interface User {
  id: number;
  name: string;
  email: string;
  isAdmin: boolean;
}

// Cách sử dụng:
const myUser: User = {
  id: 1,
  name: "Tuan Minh",
  email: "minh@example.com",
  isAdmin: true
};
```

- Tính năng mở rộng:
  - Optional properties(?): đánh dấu 1 trường có thể có hoặc ko
  - Readonly properties(readonly): Chỉ cho phép đọc, không cho phép sửa sau khởi tạo.
- Interface lồng nhau:

```
interface Address {
  street: string;
  city: string;
}

interface Customer {
  id: number;
  name: string;
  address: Address; // Sử dụng interface đã định nghĩa
ở trên
}
```

ii. Sự khác biệt giữa type và interface (khi nào dùng cái nào).

- Khả năng mở rộng:

    - Interface (Declaration Merging): Bạn có thể định nghĩa cùng một interface nhiều lần ở các nơi khác nhau, TypeScript sẽ tự động gộp chúng lại. Điều này cực kỳ hữu ích khi bạn viết thư viện hoặc muốn mở rộng các định nghĩa có sẵn.
    - Type (Intersection): Một khi đã định nghĩa type, bạn không thể khai báo lại nó để thêm thuộc tính. Bạn phải dùng dấu & để tạo một type mới dựa trên cái cũ.
- **Sự linh hoạt về kiểu dữ liệu**
    - **Type (Linh hoạt hơn):** type có thể định nghĩa cho mọi thứ: kiểu nguyên thủy (string, number), Union types, Tuples, hoặc Map types.
    - **Interface (Chuyên cho Object):** Chỉ dùng để định nghĩa cấu trúc của Object hoặc Class.
- **Hiệu năng kiểm tra kiểu (Type Checking)**
    - Về mặt kỹ thuật, TypeScript xử lý interface nhanh hơn một chút so với type (đặc biệt là khi sử dụng các phép giao &). Interface tạo ra một "bản đồ" cấu trúc rõ ràng giúp trình biên dịch tối ưu hóa tốt hơn trong các dự án lớn.

Tình huống	Nên dùng	Lý do
Định nghĩa Props cho Component	interface	Phổ biến nhất, dễ đọc, dễ mở rộng nếu cần.
Định nghĩa API Response	interface	Cấu trúc Object rõ ràng, hỗ trợ tốt cho việc kế thừa.
Dùng cho Union hoặc Tuple	type	Interface không hỗ trợ các kiểu dữ liệu này.
Viết thư viện (Library)	interface	Cho phép người dùng thư viện của bạn mở rộng (merge) thêm dữ liệu.
Các logic phức tạp (Mapped types)	type	Có khả năng biến đổi kiểu dữ liệu mạnh mẽ hơn.

iii. Định nghĩa Array các Object (ví dụ: User[]).

Khi gọi API để lấy một danh sách bài viết hoặc sản phẩm, bạn sẽ dùng cú pháp mảng:
VD:

```
interface Post {
  userId: number;
  id: number;
  title: string;
  body: string;
}

// Định nghĩa một mảng các bài viết
const postList: Post[] = [
```

```
  { userId: 1, id: 1, title: "Học Next.js", body: "Nội dung
bài viết..." },
  { userId: 1, id: 2, title: "Học TypeScript", body: "Nội
dung bài viết..." }
];
```

b. TypeScript trong React:

i. Định nghĩa kiểu cho Props của Component.

- Định nghĩa cơ bản
  - Thông thường, hay sử dụng interface để định nghĩa kiểu cho props

```
interface WelcomeProps {
  name: string;
  age: number;
  isMember?: boolean; // Thuộc tính không bắt buộc
}
```

- Định nghĩa kiểu cho các props đặc biệt:
  - Trong thực tế, props không chỉ là string hay là number mà còn có thể là các hàm hoặc các đoạn mã HTML.
    - **React.ReactNode**: Dùng cho children (có thể là văn bản, thẻ HTML, hoặc các Component khác).
    - **Hàm (Functions)**: Định nghĩa kiểu cho các sự kiện như onClick.
    - **CSS Styles**: Dùng React.CSSProperties.

      interface ButtonProps {

```
interface ButtonProps {
 label: string;
 onClick: () => void;            // Hàm không
    nhận tham số, không trả về giá trị
 children: React.ReactNode;      // Nội dung lồng
    bên trong Component
 style?: React.CSSProperties;    // Kiểu cho
    inline-style
}
```

- Kết hợp interface và type:
  - Sử dụng khi dự án trở nên phức tạp hơn

```
// Dùng type để giới hạn các giá trị cụ thể
type ProjectStatus = "Draft" | "In Progress" |
"Completed";

interface ProjectCardProps {
  title: string;
  status: ProjectStatus; // Chỉ được phép là 1 trong 3
giá trị trên
  onDelete: (id: string) => void; // Hàm nhận vào id
kiểu string
}
```

ii. *Kiểu dữ liệu cho các sự kiện (Event types như onClick, onChange).*

- Khi làm việc với các thẻ HTML trong React, định nghĩa đúng kiểu sự kiện giúp truy cập được các thuộc tính như event.target.value 1 cách chính xác mà không bị lỗi typescript.
  - **Sự kiện Click chuột:** Sử dụng React.MouseEvent<T>.
  - **Sự kiện Thay đổi Input:** Sử dụng React.ChangeEvent<T> (phổ biến nhất cho việc lấy dữ liệu từ form).
  - **Sự kiện Submit Form:** Sử dụng React.FormEvent.
- Các kiểu dữ liệu nâng cao:
  - **Union Types (|):** Cho phép một biến có thể nhận nhiều kiểu dữ liệu khác nhau (ví dụ: string | number).
  - **Optional Properties (?):** Đánh dấu các thuộc tính không bắt buộc phải có trong một Object.
  - **Generics cơ bản (<T>):** Đây là "vũ khí" cực mạnh khi viết các hàm gọi API dùng chung, giúp tái sử dụng code cho nhiều loại dữ liệu khác nhau.

2. Kiến trúc NEXT.js App router

a. Cấu trúc Routing:

i. ***File-based Routing:*** *page.tsx, layout.tsx.*

- Trong thư mục app, mỗi thư mục con đại diện cho một phân đoạn của URL. Tuy nhiên, một thư mục chỉ trở thành một trang công khai nếu nó chứa file page.tsx.
- **page.tsx**: Là file định nghĩa giao diện duy nhất cho lộ trình (route) đó.
- **layout.tsx**: Là file định nghĩa giao diện chung cho lộ trình đó và tất cả các lộ trình con bên trong nó (ví dụ: Header, Footer, Sidebar). Layout sẽ không bị render lại khi chuyển giữa các trang con, giúp tối ưu hiệu năng.

i. ***Nested Routes:*** *Tạo thư mục lồng nhau để tạo URL phân cấp.*

- Để tạo một đường dẫn dạng /dashboard/settings, bạn chỉ cần tạo cấu trúc thư mục lồng nhau:
- Tạo thư mục dashboard.
- Bên trong dashboard, tạo thư mục settings.
- Trong settings, tạo file page.tsx.
- Lúc này, nếu dashboard có một file layout.tsx, thì trang settings sẽ tự động được bọc bởi layout của dashboard.

i. ***Dynamic Routes:*** *Sử dụng [id] hoặc [slug] để tạo trang chi tiết sản phẩm/bài viết.*

Khi không biết trước tên chính xác của URL (ví dụ: ID của một dự án startup hoặc slug của một bài viết), bạn sử dụng dấu ngoặc vuông [].

- **Cách tạo**: Tạo thư mục có tên [id] hoặc [slug].
- **Ví dụ**: Thư mục app/projects/[id]/page.tsx sẽ khớp với các URL như /projects/1, /projects/abc, v.v.

B. Thành phần Layout:

i. Root Layout:

Mọi dự án Next.js dùng App Router đều **bắt buộc** phải có một file layout.tsx nằm ở thư mục app ngoài cùng.

- Đây là nơi duy nhất được phép định nghĩa cấu trúc nền tảng của HTML như thẻ <html> và <body>.
- Mục đích chính của Root Layout là **cấu hình HTML, Body và các font chữ dùng chung** cho toàn bộ ứng dụng. Bạn cũng có thể dùng nó để chứa Navbar hoặc Footer nếu muốn chúng xuất hiện ở mọi trang.

ii. Local Layout:

Khi dự án lớn lên, bạn sẽ có những khu vực cần giao diện hoàn toàn khác nhau.

- **Cách hoạt động:** có thể tạo thêm các file layout.tsx bên trong các thư mục con. Nó sẽ tạo ra cấu trúc lồng nhau (Nested Layouts).
- **Ứng dụng thực tế:** Đây là **cách tạo Header/Sidebar riêng cho từng phân khu**.

  Ví dụ, có thể tạo một layout có Sidebar màu tối cho **khu vực Admin**, trong khi vẫn giữ nguyên giao diện tươi sáng cho **khu vực User**. Khi người dùng điều hướng bên trong khu vực Admin, phần Sidebar đó sẽ không bị tải lại.

i. Template vs Layout:

Cả layout.tsx và template.tsx đều dùng để bọc các component con, nhưng cách chúng xử lý việc chuyển trang (navigation) lại hoàn toàn khác nhau. Việc hiểu rõ **khi nào cần render lại toàn bộ component** là rất quan trọng:

Đặc điểm	Layout (layout.tsx)	Template (template.tsx)
Hành vi khi chuyển trang	Không bị render lại (Chỉ nội dung bên trong thay đổi).	Bị hủy (unmount) và render lại toàn bộ component.
Trạng thái (State)	Giữ nguyên. (Ví dụ: Bạn đang gõ dở text ở thanh tìm kiếm, chuyển trang text vẫn còn).	Reset hoàn toàn về ban đầu.
Hiệu ứng (Animation)	Không chạy lại.	Chạy lại hiệu ứng xuất hiện (Enter animation) mỗi lần chuyển trang.

3. Server và client component

a. Server Components (Mặc định):

Theo mặc định, tất cả components trong thư mục app của next.js đều là server component

- **Ưu điểm:** Cực kỳ tốt cho SEO và giúp tăng tốc độ tải trang ban đầu. Do trình duyệt không phải tải xuống và thực thi các đoạn mã JavaScript nặng nề, người dùng sẽ thấy nội dung trang web gần như ngay lập tức.
- **Hạn chế:** Vì chạy trên server (không có môi trường trình duyệt DOM), **không thể** dùng được các Hook của React (như useState, useEffect) hay các sự kiện tương tác người dùng (như onClick, onChange).

b. Client Components:

Đây là các component hoạt động giống hệt như cách viết React thuần túy trước đây. Chúng chạy mã JavaScript trực tiếp trên trình duyệt của người dùng.

- **Cách sử dụng:** Để biến một component thành Client Component, phải khai báo directive 'use client' ở dòng đầu tiên cùng của file.
- **Khi nào bắt buộc phải dùng:** cần đến chúng khi xây dựng các thành phần có tính tương tác cao (Interactive UI) như Biểu mẫu (Form), thanh trượt (Slider), nút bấm thả tim, hoặc các modal pop-up.

a. Phối hợp Component:

Nguyên tắc "<u>Đẩy Client Component ra rìa</u>" (Moving Client Components Down the Tree / Leaf Components)

- Để giảm thiểu lượng JavaScript phải gửi xuống trình duyệt, nên giữ phần lớn giao diện là Server Component và chỉ chuyển đổi các phần thực sự cần tương tác thành Client Component.
- Ví dụ: Nếu có một giao diện bố cục (Layout) chứa các yếu tố tĩnh (như logo, danh sách liên kết) và một thanh tìm kiếm có tính tương tác, không thêm 'use client' vào toàn bộ Layout. Thay vào đó, hãy tách phần tương tác thành một Client Component độc lập (ví dụ: <SearchBar />) và giữ Layout chính là Server Component.

Cách lồng ghép hiệu quả:

- Hoàn toàn có thể lồng (import) một Client Component vào bên trong một Server Component.
- Tuy nhiên, chiều ngược lại (import Server Component trực tiếp vào Client Component) sẽ biến Server Component đó thành Client Component.
- Để giải quyết, phải truyền Server Component dưới dạng children props.

4. Data Fetching & API Integration

a. Lấy dữ liệu (Fetching):

Next.js cung cấp hai phương pháp chính để giao tiếp với hệ thống dữ liệu: sử dụng `fetch` API trên Server Component và sử dụng thư viện bên thứ ba (như `axios`) trên Client Component.

Native fetch với async/await (Server Components)

Cho phép gọi API trực tiếp trên server, ẩn giấu logic và API Key khỏi trình duyệt

```
// app/projects/page.tsx
export default async function ProjectList() {
  // Lấy dữ liệu trực tiếp bằng fetch
  const res = await fetch('https://api.example.com/projects');
  const projects = await res.json();

  return (
    <ul>
      {projects.map((p: any) => <li key={p.id}>{p.name}</li>)}
    </ul>
  );
}
```

Chiến lược Caching với fetch

Cơ chế quản lý bộ nhớ đệm (Cache) và tự động làm mới (Revalidate) của Next.js.

Tùy chọn fetch	Cú pháp	Khi nào sử dụng?
Static (Mặc định)	{ cache: 'force-cache' }	Dữ liệu tĩnh, ít thay đổi (Blog, Giới thiệu).
Dynamic (Real-time)	{ cache: 'no-store' }	Dữ liệu thay đổi liên tục, cần mới nhất ở mỗi request.
ISR (Revalidate)	{ next: { revalidate: 60 } }	Dữ liệu cần cập nhật ngầm định kỳ (ví dụ: mỗi 60 giây).

Sử dụng Axios (Client Components)

Sử dụng cho các dự án cần cấu hình phức tạp (Interceptors, Timeout, gắn Token)

```
// app/components/CreateProjectForm.tsx
'use client'
import axios from 'axios';

export default function Form() {
  const handleSubmit = async () => {
    try {
      // Axios tự động parse JSON
      const res = await axios.post('/api/projects', { name:
"Dự án A" });
      console.log(res.data);
    } catch (error) {
      console.error("Lỗi khi tạo dự án", error);
    }
  }
  return <button onClick={handleSubmit}>Tạo Dự Án</button>;
}
```

b. Xử lý trạng thái API:

Next.js sử dụng file-system conventions để tự động bao bọc các tuyến đường bằng Suspense và Error Boundaries.

Trạng thái chờ (loading.tsx)

Tạo file loading.tsx để hiển thị UI thay thế (Skeleton/Spinner) ngay lập tức trong khi Server Component đang chờ gọi API.

```
// app/projects/loading.tsx
export default function Loading() {
  return <div className="spinner">Đang tải dữ liệu dự
án...</div>;
}
```

Xử lý lỗi (error.tsx)

Tạo file error.tsx để bắt các lỗi phát sinh trong quá trình lấy dữ liệu và hiển thị giao diện khôi phục.

Lưu ý: File này bắt buộc phải là Client Component.

```
// app/projects/error.tsx
'use client' // Error components phải chạy ở client

export default function Error({ error, reset }: { error:
Error, reset: () => void }) {
  return (
```

```
      <div>
        <h2>Đã xảy ra lỗi hệ thống!</h2>
        <button onClick={() => reset()}>Thử lại</button>
      </div>
    );
  }
```

c. Route Handlers (xây dựng API nội bộ):

Tạo các API Endpoints trực tiếp trong dự án Next.js thay vì xây dựng server Node.js riêng biệt.
Cấu trúc và Cú pháp:
Tạo file route.ts bên trong thư mục app/api/... và export các hàm HTTP tương ứng (GET, POST, PUT, DELETE).

```
// app/api/projects/route.ts
import { NextResponse } from 'next/server';

// Xử lý request GET [cite: 69]
export async function GET() {
  const data = [ { id: 1, name: "Dự án Khởi nghiệp" } ];
  return NextResponse.json(data);
}

// Xử lý request POST [cite: 69]
export async function POST(request: Request) {
  const body = await request.json();
  // ... Logic lưu database (Prisma/Mongoose) ...
  return NextResponse.json({ message: "Thành công!", data:
body }, { status: 201 });
}
```